

1. Наблюдатель + то как работает прога (в крации)

Контроллер получает данные из модели

```
def initialize(view)
  @view = view
  @student_list = StudentsListYAML.new('studentlist.yaml')
  @current_page = 1
  @page_size = 10
  @filters = {}
end

def refresh_data
  @current_data_list = @student_list.get_k_n_student_short_list(@page_size, @current_page, @current_data_list)
  @current_data_list.add_observer(@view)
  @current_data_list.notify_observers
end
```

передаёт их через DataListStudentShort который уведомляет View, что таблицу нужно обновить

```
def add_observer(observer)
  @observers << observer
end

def notify_observers
  @observers.each do |observer|
    observer.on_data_changed(self)
  end
end

def on_data_changed(data_list)
  table_data = data_list.get_data
  column_names = data_list.get_names

  set_table_params(column_names, table_data.rows)
  set_table_data(table_data)
  update_pagination_info
end
```

Это полностью паттерн Наблюдатель: модель уведомляет View об изменениях данных.

2. Модель, наблюдаемый и наблюдатель

Модели: StudentsListYAML + Student + DataListStudentShort

наблюдаемый объект: дата лист студент шорт (методы добавления наблюдателя и оповещения)

наблюдатель: вью (реагирует на изменения с помощью метода on data changed)

3. Делегирование — процесс, при котором один объект передает выполнение работы другому объекту

Паттерн «Стратегия» является частным случаем делегирования, при котором контекст делегирует выполнение алгоритма одному из взаимозаменяемых объектов-стратегий, выбираемых во время выполнения программы.

делегирование: View передаёт задачу (что делать при нажатии кнопки) контроллеру.

4. Паттерн стратегия

Паттерн стратегия — это паттерн, при котором один объект делегирует выполнение определённой задачи внешнему объекту, реализующему конкретный алгоритм (стратегию). Алгоритмы инкапсулируются в отдельные классы или методы, что позволяет изменять поведение объекта во время выполнения программы без изменения его кода.

```
class Ex
  attr_accessor :num1, :num2

  def initialize(num1, num2)
    @num1 = num1
    @num2 = num2
  end

  def number
    @num1 + @num2
  end
end
```

```
class Ex2
  attr_accessor :num1, :num2

  def initialize(num1, num2)
    @num1 = num1
    @num2 = num2
  end

  def number
    @num1 * @num2
  end
end
```

```
class Itog
  attr_accessor :strategy
  def initialize(ex)
    @strategy = ex
  end

  def number
    @strategy.number
  end
end

one = Ex.new(3,5)
two = Ex2.new(3,5)
three = Itog.new(two)
puts three.number
```

Контекст делегирует выполнение алгоритма объекту-стратегии.

5. Паттерн наблюдатель

Паттерн наблюдатель - паттерн, при котором один объект наблюдает и реагирует на изменения другого объекта. Наблюдаемый объект содержит в себе список наблюдателей и метод оповещения наблюдателей через единый интерфейс при каких-либо изменениях

6. MVC

MVC (Model-View-Controller) — это архитектурный паттерн (шаблон проектирования), который разделяет логику приложения на три взаимосвязанных компонента. Его главная цель — отделить данные, представление и бизнес-логику, чтобы упростить разработку, тестирование и поддержку кода.

Обычно он выглядит так:

Пользователь -> Controller -> Model -> Controller -> View -> Пользователь

7. Агрегация + Ассоциация + Наследование

Ассоциация — это связь, при которой один объект использует другой объект для выполнения своих операций.

Агрегация — это связь «часть–целое», при которой часть может существовать независимо от целого. (один объект хранит ссылку на другой, но не управляет его жизнью)

Наследование — это механизм объектно-ориентированного программирования, позволяющий одному классу расширять функциональность другого. (один класс наследует функционал другого класса)

Задай вопрос:

1. Передаётся в метод? → Ассоциация
2. Хранится в поле? → Агрегация
3. Создаётся внутри? → Композиция
4. < / <|-- ? → Наследование